

AI Upskilling Platform

Cricket-Personalised Lessons — Interest-Based Learning Demo

CONTENTS

-  Reading Java Mastery: Introduction & Core Concepts
-  Exercise Python for Everyone: Hands-on Exercise
-  Quiz React 19 & Ecosystem: Quiz
-  Reading System Design: Deep Dive & Advanced Patterns
-  Project AWS Solutions Architect: Capstone Mini Project

Interest:  Cricket - Generated 2026-05-29 - Powered by Claude claude-sonnet-4-6

Java Mastery: Introduction & Core Concepts

Topic: Java Mastery

Overview

Just as a cricket match has structured innings with clear rules for every player type, Java uses object-oriented programming to model real-world entities with well-defined contracts. A batsman knows how to bat, a bowler knows how to bowl, and an umpire enforces the laws, all without needing to know each other's internal mechanics. Java captures exactly this kind of structure: each object encapsulates its own state and behaviour while exposing a clean public interface. This discipline is why Java dominates enterprise systems. Spring Boot powers banking back-ends and microservices, Android runs billions of phones, and large financial institutions trust the JVM for its stability, strong typing, mature tooling, and decades of battle-tested libraries. Like a well-drilled cricket team, Java code is predictable, maintainable, and scales gracefully from a club scoreboard to a global tournament platform serving millions of fans simultaneously.

Core Concepts

In Java, a class is a blueprint, much like the role description for a cricketer. The `CricketPlayer` class defines what every player has (a name, a jersey number) and what every player can do. Inheritance lets you build a hierarchy: `Batsman` and `Bowler` both extend `CricketPlayer`, inheriting common traits while specialising behaviour. Interfaces model cricket roles independent of the inheritance tree. A single player can be both a `Batsman` and a `Captain`, exactly like a class implementing multiple interfaces, because `Captain` is a contract of responsibilities (leading the field, making the toss call) rather than a kind of player. Polymorphism is the substitution principle in action: when the twelfth man replaces an injured fielder, the match continues calling the same `fieldBall()` method, and the right behaviour runs for whoever is on the pitch. Your code holds references of type `CricketPlayer` but the actual object decides which overridden method executes at runtime.

java

```
import java.util.ArrayList;
import java.util.List;

abstract class CricketPlayer {
    protected final String name;
    protected final int jerseyNumber;

    protected CricketPlayer(String name, int jerseyNumber) {
        this.name = name;
        this.jerseyNumber = jerseyNumber;
    }

    // Abstract method: every player type must define its own role.
    public abstract String getRole();

    public String getName() { return name; }
}

// A role contract, independent of the class hierarchy.
interface Captain {
    String callToss();
}

class Batsman extends CricketPlayer implements Captain {
    private final int battingAverage;

    public Batsman(String name, int jerseyNumber, int battingAverage) {
        super(name, jerseyNumber);
        this.battingAverage = battingAverage;
    }

    @Override
    public String getRole() {
        return name + " is a Batsman (avg " + battingAverage + ")";
    }

    @Override
    public String callToss() {
        return name + " calls Heads as captain";
    }
}
```

```

}

class Bowler extends CricketPlayer {
    private final double economyRate;

    public Bowler(String name, int jerseyNumber, double economyRate) {
        super(name, jerseyNumber);
        this.economyRate = economyRate;
    }

    @Override
    public String getRole() {
        return name + " is a Bowler (economy " + economyRate + ")";
    }
}

public class MatchLineup {
    public static void main(String[] args) {
        List<CricketPlayer> lineup = new ArrayList<>();
        lineup.add(new Batsman("Virat Kohli", 18, 58));
        lineup.add(new Bowler("Jasprit Bumrah", 93, 4.6));

        // Polymorphism: same call, subtype-specific behaviour at runtime.
        for (CricketPlayer player : lineup) {
            System.out.println(player.getRole());
        }

        // A Batsman who is also a Captain.
        Batsman skipper = (Batsman) lineup.get(0);
        System.out.println(skipper.callToss());
    }
}

```

The abstract method `getRole()` in `CricketPlayer` is a promise without an implementation: the class declares that every player must define a role, but refuses to say how, so you cannot instantiate a bare `CricketPlayer`. `Batsman` and `Bowler` each provide their own `@Override` of `getRole()`, giving subtype-specific behaviour. When you loop over the `List<CricketPlayer>` and call `player.getRole()`, the compiler only knows the reference is a `CricketPlayer`, but the JVM performs dynamic dispatch: it consults the actual object's method table (vtable) at runtime and invokes the correct overridden method. That is why Kohli prints batsman text and Bumrah prints bowler text from a single call site.

How It Works Under the Hood

When you compile `MatchLineup.java` with `javac`, you get `.class` files containing platform-independent bytecode, the cricket equivalent of a standardised scorecard format every ground understands. At runtime the class loader subsystem reads each `.class` file, verifies the bytecode, and loads classes lazily, so `Bowler` is only loaded the first time `Bumrah` is created. Newly created objects, such as a `Scoreboard` tracking live runs, live on the heap and are shared and garbage-collected when no longer referenced. Each method invocation pushes a frame onto the thread's stack holding local variables and the operand stack; when `updateScore()` returns, its frame pops off. Initially the JVM interprets bytecode, but HotSpot's JIT compiler watches for hot methods (a scoreboard refresh called every ball) and compiles them to optimised native machine code, so a long innings runs progressively faster as the code warms up.

java

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

// Generic scorecard bounded to cricket players only.
class ScoreCard<T extends CricketPlayer> {
    private final List<T> players = new ArrayList<>();
    private final Map<String, Integer> runTally = new HashMap<>();

    public void record(T player, int runs) {
        players.add(player);
        runTally.merge(player.getName(), runs, Integer::sum);
    }

    public Map<String, Integer> getRunTally() { return runTally; }

    // Sort players by runs scored, highest first.
    public List<T> sortedByRuns() {
        List<T> sorted = new ArrayList<>(players);
        sorted.sort(Comparator.comparingInt(
            (T p) → runTally.getOrDefault(p.getName(), 0)).reversed());
        return sorted;
    }

    public static void main(String[] args) {
        ScoreCard<Batsman> indiaTop = new ScoreCard<>();
        indiaTop.record(new Batsman("Rohit Sharma", 45, 49), 67);
        indiaTop.record(new Batsman("Shubman Gill", 77, 47), 92);

        for (Batsman b : indiaTop.sortedByRuns()) {
            System.out.println(b.getName() + ": "
                + indiaTop.getRunTally().get(b.getName()) + " runs");
        }
    }
}
```

💡 Pro Tip: When modelling a player's roles such as BATSMAN, BOWLER, WICKETKEEPER, and CAPTAIN, reach for EnumSet instead of manual bitwise int flags. EnumSet is internally backed by a single long bit vector, so it is as fast and compact as hand-rolled flags but fully type-safe and readable: EnumSet.of(Role.BATSMAN, Role.CAPTAIN). A non-obvious performance insight: the JVM's HotSpot JIT only optimises a method after it crosses an invocation threshold (roughly 10,000 calls), so micro-benchmarking your scorecard logic on the very first innings gives misleading 'cold' numbers. Always warm up the JIT with a few thousand iterations before trusting timing results.

Common Patterns & Best Practices

Three patterns recur constantly in cricket-domain Java. The Builder pattern assembles complex Match objects step by step, so you can set teams, venue, and format fluently and only produce an immutable Match once everything is valid. This is crucial because a match with no teams is nonsense: Builder.build() can reject invalid state before any object escapes into the system, preventing half-constructed matches from corrupting your scoreboard. The Factory pattern centralises creation of player types: a PlayerFactory decides whether to return a Batsman, Bowler, or AllRounder based on a role string from a team sheet, keeping construction logic in one place. The Strategy pattern swaps scoring rules at runtime: a T20ScoringStrategy and a TestScoringStrategy implement a common interface, so the same engine computes results differently for an IPL fixture versus a five-day Test, without sprawling if-else chains.

java

```
public final class Match {
    private final String teamA;
    private final String teamB;
    private final String venue;
    private final String format;

    private Match(Builder builder) {
        this.teamA = builder.teamA;
        this.teamB = builder.teamB;
        this.venue = builder.venue;
        this.format = builder.format;
    }

    public static class Builder {
        private String teamA;
        private String teamB;
        private String venue = "TBD";
        private String format = "T20";

        public Builder teams(String a, String b) {
            this.teamA = a;
            this.teamB = b;
            return this;
        }

        public Builder venue(String venue) {
            this.venue = venue;
            return this;
        }

        public Builder format(String format) {
            this.format = format;
            return this;
        }

        public Match build() {
            if (teamA == null || teamB == null) {
                throw new IllegalStateException(
                    "A match needs two teams");
            }
        }
    }
}
```



```

        if (teamA.equals(teamB)) {
            throw new IllegalStateException(
                "A team cannot play itself");
        }
        return new Match(this);
    }
}

public static void main(String[] args) {
    Match finalMatch = new Match.Builder()
        .teams("India", "Australia")
        .venue("Narendra Modi Stadium")
        .format("ODI World Cup Final")
        .build();

    System.out.println("Match created at " + finalMatch.venue);
}
}

```

 **Warning:** Accessing player statistics that may be absent is a classic `NullPointerException` trap. If you write `int sr = player.getStats().getStrikeRate()` and a debutant has no stats yet, `getStats()` returns null and your scoreboard crashes mid-over. Never let null leak through your API. Return `Optional<Integer>` for values that might not exist and force callers to handle the empty case: `int runs = player.getRuns().orElse(0)`. This converts a lurking runtime crash into an explicit, compile-time-visible decision about what to do when a player has not yet faced a ball.

Real-World Application

Platforms like ESPNcricinfo, Dream11, and the BCCI's official apps lean heavily on Java and Spring Boot to expose match data APIs, push real-time ball-by-ball scoring, and crunch player analytics. A Spring Boot service models each Match, Innings, and Player as domain objects, exposes REST endpoints for scorecards, and uses dependency injection to wire repositories and services cleanly. The very same OOP principles you saw, encapsulation, polymorphism, and well-defined interfaces, are what let these systems scale from a single club fixture to millions of concurrent fans refreshing scores during an India-Pakistan World Cup clash without rewriting the core domain model.

KEY POINTS

- Encapsulation hides a Batsman's internal stats behind methods, so you change the scoring formula without breaking any code that reads runs.
- Abstract classes like `CricketPlayer` share common state and force subtypes to implement role-specific behaviour, while still preventing direct instantiation of a generic player.
- Interfaces such as `Captain` model capabilities orthogonal to the class tree, letting one Batsman also lead the side without multiple class inheritance.
- Polymorphic dispatch resolves `player.getRole()` at runtime via the object's method table, enabling one loop over `List<CricketPlayer>` to produce subtype-specific output.
- Favour immutability for identity: a final `Match` built through a `Builder` cannot drift into an invalid two-teams-missing state after construction completes.
- Use `Optional<Integer>` rather than `null` for stats that may be absent, turning silent `NullPointerExceptions` into explicit, well-handled empty cases.

Q: A `Bowler` and `Batsman` both extend `CricketPlayer`. You store them in `List<CricketPlayer>` and call `getRole()`. Which Java mechanism enables different outputs for each subtype?

- A** Method overloading resolved by the compiler based on argument types
- B** Runtime polymorphism via dynamic dispatch using the object's method table
- C** Static binding that fixes the method at compile time based on the reference type
- D** Generics type erasure substituting the concrete type at runtime

Explanation: The reference type is `CricketPlayer`, but Java uses dynamic (late) binding for overridden instance methods. At runtime the JVM inspects the actual object, `Batsman` or `Bowler`, and consults its virtual method table to invoke the correct `getRole()` override. Overloading (A) is compile-time and based on signatures, static binding (C) would ignore the real object, and type erasure (D) concerns generics, not method dispatch.

Q: Your cricket app needs to track player statistics that change frequently but player identity (name, ID) never changes. Which Java design best models this?

- A** Make the entire Player class mutable so any field can change at any time
- B** Make the entire Player class immutable, recreating it on every score update
- C** Model immutable identity fields (final name, ID) separately from a mutable stats object the player references
- D** Use only static fields so all players share one global statistics record

Explanation: Identity should be immutable: a player's name and ID are set once and marked final, which makes them safe to use as map keys and share across threads. Frequently changing data like runs and strike rate belongs in a separate mutable Stats object. Fully mutable (A) risks corrupting identity, fully immutable (B) wastes allocations on every ball, and static fields (D) would wrongly merge every player's stats into one shared record.

Python for Everyone: Hands-on Exercise

Topic: Python for Everyone

What You'll Build

You will build a command-line cricket scorecard tracker in pure Python. The app lets you create a match between two teams, record batting entries (batsman, runs, balls faced, fours, sixes, and how they were dismissed) and bowling figures (bowler, overs, maidens, runs conceded, wickets). It then calculates real cricket statistics such as strike rate, economy rate, and run rate, and persists everything to a JSON file so you can resume a match later. By the end you will have a working tool that turns raw deliveries into a neatly formatted scorecard, the perfect project for a cricket fan learning Python through a domain they already love.

Prerequisites

KEY POINTS

- Python 3.8 or newer installed (the project uses dataclasses and f-strings).
- Comfort with built-in collections, especially dictionaries and lists for holding batting and bowling rows.
- A basic grasp of object-oriented thinking, since each scorecard row maps to a class.
- pip available for installing optional tooling, though this project needs no external dependencies.
- A terminal where you can run python3 and create directories.

Setup & Project Structure

The layout is deliberately minimal so you can focus on the cricket logic rather than packaging. A single `scorecard.py` file holds the data models, calculations, and the command-line interface. A `data/` directory stores one JSON file per match, named after the two teams, acting as your lightweight database. There are no external dependencies at all, everything uses Python's standard library: `dataclasses`, `json`, and `argparse`.

```
bash
```

```
mkdir cricket_scorecard && cd cricket_scorecard
mkdir data
touch scorecard.py

# Resulting tree structure:
# cricket_scorecard/
# |-- scorecard.py      # data models, logic, and CLI
# |-- data/            # one JSON file per match
#   |-- india_vs_australia.json
```

Step 1 — Foundation: Data Models

We model each row of a scorecard with a dataclass. A `BattingEntry` is exactly one batting line, and a `BowlingEntry` is exactly one bowling line, just like printed rows on a real scorecard. The `Innings` dataclass is the container that holds the two teams plus lists of batting and bowling entries, and knows how to serialise itself to JSON and load back. Think of the dataclass as a single scorecard row: it auto-generates the constructor and repr so you write almost no boilerplate.

python

```
from __future__ import annotations
import json
import os
from dataclasses import dataclass, field, asdict
from typing import List

@dataclass
class BattingEntry:
    batsman: str
    runs: int = 0
    balls: int = 0
    fours: int = 0
    sixes: int = 0
    dismissal: str = "not out"
    is_out: bool = False

@dataclass
class BowlingEntry:
    bowler: str
    overs: float = 0.0
    maidens: int = 0
    runs_conceded: int = 0
    wickets: int = 0

@dataclass
class Innings:
    team1: str
    team2: str
    batting: List[BattingEntry] = field(default_factory=list)
    bowling: List[BowlingEntry] = field(default_factory=list)

    def save(self, path: str) → None:
        with open(path, "w") as f:
            json.dump(asdict(self), f, indent=2)

    @classmethod
    def load(cls, path: str) → "Innings":
```

```
with open(path) as f:
    data = json.load(f)
innings = cls(team1=data["team1"], team2=data["team2"])
innings.batting = [BattingEntry(**b) for b in data["batting"]]
innings.bowling = [BowlingEntry(**b) for b in data["bowling"]]
return innings
```

Step 2 — Core Logic: Calculations & Operations

Now we add the cricket maths. Strike rate is runs divided by balls faced times 100. Economy rate is runs conceded divided by overs bowled. Run rate is total team runs divided by overs bowled. We also add helper operations: `add_batsman` and `add_bowler` append new rows, `get_total_runs` and `get_total_wickets` aggregate the innings, and `format_scorecard` renders everything into a readable block of text, the way a broadcast scorecard would look.

```

def add_batsman(self, entry: BattingEntry) → None:
    self.batting.append(entry)

def add_bowler(self, entry: BowlingEntry) → None:
    self.bowling.append(entry)

def get_total_runs(self) → int:
    return sum(b.runs for b in self.batting)

def get_total_wickets(self) → int:
    return sum(1 for b in self.batting if b.is_out)

@staticmethod
def strike_rate(entry: BattingEntry) → float:
    return round(entry.runs / entry.balls * 100, 2) if entry.balls else 0.0

@staticmethod
def economy(entry: BowlingEntry) → float:
    return round(entry.runs_conceded / entry.overs, 2) if entry.overs else 0.0

def get_run_rate(self, overs_bowled: float) → float:
    return round(self.get_total_runs() / overs_bowled, 2) if overs_bowled else 0.0

def format_scorecard(self) → str:
    lines = [f"{self.team1} vs {self.team2}", "=" * 40, "BATTING"]
    lines.append(f"{'Batsman':20}{ 'R':>4}{ 'B':>4}{ '4s':>4}{ '6s':>4}{ 'SR':>8}")
    for b in self.batting:
        lines.append(
            f"{b.batsman:20}{b.runs:>4}{b.balls:>4}"
            f"{b.fours:>4}{b.sixes:>4}{self.strike_rate(b):>8}"
        )
    total = self.get_total_runs()
    wkts = self.get_total_wickets()
    lines.append(f"TOTAL: {total}/{wkts}")
    lines.append("")
    lines.append("BOWLING")
    lines.append(f"{'Bowler':20}{ 'O':>5}{ 'M':>4}{ 'R':>4}{ 'W':>4}{ 'Econ':>8}")
    for w in self.bowling:
        lines.append(
            f"{w.bowler:20}{w.overs:>5}{w.maidens:>4}"

```



```
        f"{w.runs_conceded:>4}-{w.wickets:>4}-{self.economy(w):>8}"  
    )  
    return "\n".join(lines)
```

Step 3 — Integration: CLI Interface

The command-line interface uses argparse subcommands so each action reads like a cricket instruction. `new-match` creates a fresh match file from `--team1` and `--team2`, `add-bat` records a batting entry, `add-bowl` records bowling figures, and `show` prints the formatted scorecard. We use the match name (derived from the two teams, lowercased and joined) as the JSON file identifier so every command knows which match file to read and write.

python

```
import argparse

DATA_DIR = "data"

def match_path(match: str) → str:
    return os.path.join(DATA_DIR, f"{match}.json")

def main() → None:
    parser = argparse.ArgumentParser(description="Cricket scorecard tracker")
    sub = parser.add_subparsers(dest="command", required=True)

    p_new = sub.add_parser("new-match")
    p_new.add_argument("--team1", required=True)
    p_new.add_argument("--team2", required=True)

    p_bat = sub.add_parser("add-bat")
    p_bat.add_argument("--match", required=True)
    p_bat.add_argument("--batsman", required=True)
    p_bat.add_argument("--runs", type=int, default=0)
    p_bat.add_argument("--balls", type=int, default=0)
    p_bat.add_argument("--fours", type=int, default=0)
    p_bat.add_argument("--sixes", type=int, default=0)
    p_bat.add_argument("--dismissed", default="not out")

    p_bowl = sub.add_parser("add-bowl")
    p_bowl.add_argument("--match", required=True)
    p_bowl.add_argument("--bowler", required=True)
    p_bowl.add_argument("--overs", type=float, default=0.0)
    p_bowl.add_argument("--maidens", type=int, default=0)
    p_bowl.add_argument("--runs", type=int, default=0)
    p_bowl.add_argument("--wickets", type=int, default=0)

    p_show = sub.add_parser("show")
    p_show.add_argument("--match", required=True)

    args = parser.parse_args()
    os.makedirs(DATA_DIR, exist_ok=True)
```

```

if args.command == "new-match":
    match = f"{args.team1.lower()}_vs_{args.team2.lower()}"
    innings = Innings(team1=args.team1, team2=args.team2)
    innings.save(match_path(match))
    print(f"Created match: {match}")

elif args.command == "add-bat":
    innings = Innings.load(match_path(args.match))
    innings.add_batsman(BattingEntry(
        batsman=args.batsman, runs=args.runs, balls=args.balls,
        fours=args.fours, sixes=args.sixes,
        dismissal=args.dismissed,
        is_out=(args.dismissed != "not out"),
    ))
    innings.save(match_path(args.match))
    print(f"Recorded {args.batsman}: {args.runs} ({args.balls})")

elif args.command == "add-bowl":
    innings = Innings.load(match_path(args.match))
    innings.add_bowler(BowlingEntry(
        bowler=args.bowler, overs=args.overs, maidens=args.maidens,
        runs_conceded=args.runs, wickets=args.wickets,
    ))
    innings.save(match_path(args.match))
    print(f"Recorded {args.bowler}: {args.wickets}/{args.runs}")

elif args.command == "show":
    innings = Innings.load(match_path(args.match))
    print(innings.format_scorecard())

if __name__ == "__main__":
    main()

```

Step 4 — Testing & Verification

Let's drive the tool through a realistic scenario. First create an India versus Australia match, then record India's top-order batting starting with Rohit Sharma, then enter Jasprit Bumrah's bowling figures, and finally display the full scorecard to confirm the strike rates and economy rates were computed correctly.

```
bash
```

```
python scorecard.py new-match --team1 India --team2 Australia
python scorecard.py add-bat --match india_vs_australia \
    --batsman "Rohit Sharma" --runs 67 --balls 45 --fours 8 --sixes 2
python scorecard.py add-bat --match india_vs_australia \
    --batsman "Shubman Gill" --runs 31 --balls 28 --fours 4 --sixes 0 \
    --dismissed "c Smith b Starc"
python scorecard.py add-bowl --match india_vs_australia \
    --bowler "Jasprit Bumrah" --overs 4 --maidens 1 --runs 23 --wickets 2
python scorecard.py show --match india_vs_australia
```

```
# Expected output:
```

```
# India vs Australia
```

```
# =====
```

```
# BATTING
```

# Batsman	R	B	4s	6s	SR
# Rohit Sharma	67	45	8	2	148.89
# Shubman Gill	31	28	4	0	110.71
# TOTAL:	98/1				

```
#
```

```
# BOWLING
```

# Bowler	O	M	R	W	Econ
# Jasprit Bumrah	4.0	1	23	2	5.75

 **Warning:** If a match JSON file is empty or partially written (for example the process was killed mid-save during a power cut at the ground), `json.load` will raise `json.JSONDecodeError` and crash every subsequent command. Always check the file exists with `os.path.exists` before loading, and wrap `json.load` in a `try/except json.JSONDecodeError` so you can print a friendly 'match file is corrupted or empty, please recreate it' message instead of dumping a stack trace at the scorer.

 **Extension Challenge:** Level up your tracker with one of these. 1) Add partnership tracking that records runs added between each pair of batsmen at the crease and flags century stands. 2) Render a wagon wheel of a batsman's scoring shots using simple ASCII art around a circle. 3) Integrate with a live ESPNcricinfo-style API to auto-populate ball-by-ball data for a real match instead of typing entries by hand, turning your CLI into a live score viewer.

KEY POINTS

- Dataclasses (BattingEntry, BowlingEntry, Innings) give you free constructors and repr, mapping cleanly onto scorecard rows.
- asdict plus json.dump serialises nested dataclasses to disk, and **kwargs unpacking rebuilds them on load.
- argparse subparsers model distinct CLI verbs (new-match, add-bat, add-bowl, show) with their own validated arguments.
- f-string format specifiers like {b.runs:>4} produce aligned, broadcast-style columns in the scorecard.
- Cricket statistics are simple ratios: strike rate is runs/balls*100, economy is runs/overs, run rate is total runs/overs.
- Guarding file reads against missing or corrupt JSON keeps the tool robust during real match-day use.

React 19 & Ecosystem: Quiz

Topic: React 19 & Ecosystem

Knowledge Check — React 19 & Ecosystem

This quiz tests your understanding of React hooks, rendering behaviour, state management, and performance optimisation. The applied questions are framed around building a live cricket scoring dashboard that streams real-time ball-by-ball updates, so you will reason about re-renders, memoisation, effects, and server components exactly as you would when shipping a production scoreboard during an IPL match.

Q: What is the primary purpose of React's `useCallback` hook?

- A** It memoises the result of an expensive calculation between renders
- B** It returns a memoised function reference that stays stable across renders to prevent unnecessary child re-renders
- C** It schedules a side effect to run after the component commits to the DOM
- D** It forces a component to re-render whenever the function body changes

Explanation: `useCallback` returns a memoised version of a callback whose identity only changes when its dependencies change. This matters because passing a fresh function each render to a `React.memo` child (like a `WicketAlert` button) would break the child's prop equality and force a re-render. `useMemo` (A) memoises values, `useEffect` (C) runs side effects, so (B) is correct: a stable function reference.

Q: In React 18+, what happens when you call multiple `setState()` calls inside a native event handler?

- A** Each `setState` triggers its own immediate synchronous re-render
- B** Only the last `setState` call takes effect and the others are discarded
- C** They are automatically batched into a single re-render for better performance
- D** React throws an error because multiple updates are not allowed per handler

Explanation: React 18 introduced automatic batching everywhere, including native event handlers, timeouts, and promises, not just React synthetic events. Multiple state updates within the same tick are grouped so the component re-renders once with all updates applied. Updating score, over, and wicket count together in a WebSocket handler therefore yields one render, not three. The other options describe pre-18 or incorrect behaviour.

Q: Your live cricket scoreboard shows ball-by-ball commentary. You wrap the `CommentaryFeed` component in `React.memo`. When does it still re-render?

- A** Never, `React.memo` guarantees the component will not re-render again
- B** Only when its parent re-renders, regardless of props
- C** When a prop changes by reference, for example a new array or object or function is passed even if its contents are identical
- D** Only when its internal state changes, never due to props

Explanation: `React.memo` does a shallow comparison of props. Primitives compare by value, but objects, arrays, and functions compare by reference. If the parent passes a freshly created `commentary=[...]` array or an inline `onClick` each render, the reference differs and the memoised `CommentaryFeed` still re-renders. Stabilise such props with `useMemo/useCallback` to make memoisation effective.

Q: In your cricket app, you use `useEffect` to subscribe to a `WebSocket` for live score updates. When exactly does the cleanup function run?

- A Only once, when the component is first mounted
- B Before every re-run of the effect (when dependencies change) and once when the component unmounts**
- C Immediately after the effect runs, in the same render
- D Only when the browser tab is closed

Explanation: The cleanup function returned from `useEffect` runs before the effect executes again due to a dependency change, and a final time on unmount. For a `WebSocket` this means React closes the old socket before opening a new one when the `matchId` dependency changes, and closes it when the dashboard unmounts, preventing duplicate subscriptions and memory leaks during a long innings.

Q: Your cricket dashboard has 11 batsman cards, each showing live strike rate. Every ball delivery, ALL 11 cards re-render even for unchanged batsmen. What is the correct fix?

- A Wrap each `BatsmanCard` in `React.memo` and pass stable, primitive props so unchanged cards skip re-rendering**
- B Wrap the whole list in `useCallback` to memoise the rendering function
- C Use `useMemo` to cache the strike rate number inside each card
- D Virtualise the list with `react-window` so only visible cards render

Explanation: Only the batsman who faced the ball changed, yet all cards re-render because the parent re-renders. Wrapping `BatsmanCard` in `React.memo` with stable primitive props (runs, balls) lets React skip cards whose props are unchanged. `useCallback` (B) memoises functions not components, `useMemo` (C) caches a value but still re-renders the card, and virtualisation (D) addresses long lists, not selective updates of only 11 items.

Q: You're building a cricket fantasy team app used by 500k users. Shared state: current match, user's squad, global leaderboard. Which state management fits best?

- A Put everything in a single React Context provider at the app root
- B Use server state tools (React Query/TanStack Query or RSC) for match and leaderboard data, and a light client store (Zustand) or Context for the user's local squad selections**
- C Use Redux for absolutely all state including ephemeral UI toggles
- D Lift all state to the top component and prop-drill it everywhere

Explanation: Match scores and the global leaderboard are server state, frequently changing remote data best handled by a caching/fetching library (React Query) or server components, which gives you caching, revalidation, and background refetch. The user's in-progress squad is local client state suited to Zustand or Context. A single Context (A) causes app-wide re-renders, Redux-for-everything (C) is overkill, and prop drilling (D) does not scale.

Q: In your run-chase calculator component, a `useEffect` has `[target, currentScore]` as deps. Inside it, you reference an `overs` variable from the outer component scope that's NOT in the deps array. What happens?

- A React automatically adds `overs` to the dependency array at runtime
- B The effect captures a stale closure: it keeps seeing the value of `overs` from the render when the effect last ran, leading to incorrect required run rate calculations**
- C The component crashes immediately with a missing dependency error
- D `overs` is always read fresh because closures re-read outer variables live

Explanation: Effects close over the variables from the render in which they were created. Because `overs` is omitted from the deps, the effect does not re-run when `overs` changes, so it keeps using the stale value captured last time `target` or `currentScore` changed. Your required-run-rate math will silently use an outdated `overs` figure. The lint rule warns but does not crash, and the fix is to include `overs` in the dependency array.

Q: You want to show live match commentary in a Server Component but allow users to click 'Add to Favourites' on each comment. What is the correct Next.js App Router approach?

- A** Add an onClick handler directly in the Server Component
- B** Convert the entire page to a Client Component with 'use client' at the top
- C** Keep the commentary list as a Server Component and extract the interactive 'Add to Favourites' button into a small Client Component (marked 'use client') passed as a child
- D** Use getServerSideProps to handle the click on the server

Explanation: Server Components cannot use event handlers or hooks, so the interactive button must be a Client Component marked 'use client'. The best pattern keeps the heavy, data-fetching commentary list on the server and isolates only the small FavouriteButton as a client island, minimising JavaScript sent to the browser. Marking the whole page client (B) sacrifices server rendering benefits, and getServerSideProps (D) is Pages Router, not App Router.

KEY POINTS

- useCallback memoises function references and useMemo memoises computed values, both to preserve prop equality for memoised children.
- React 18+ automatically batches state updates everywhere, so multiple setState calls in a WebSocket handler cause a single re-render.
- React.memo only skips re-renders when props are shallowly equal, so object, array, and function props must be reference-stable.
- useEffect cleanup runs before each re-run on dependency change and once on unmount, which is essential for closing live-score WebSockets.
- Match the tool to the data: server state libraries for remote match/leaderboard data, lightweight client stores for local UI like squad picks.
- Omitting a dependency causes stale closures, and interactive UI in the App Router belongs in small 'use client' islands within Server Components.

System Design: Deep Dive & Advanced Patterns

Topic: System Design

Overview

ESPNcricinfo serves more than 50 million concurrent users during a marquee India versus Pakistan World Cup match, all refreshing scores every few seconds and expecting ball-by-ball updates within a heartbeat. How do you design a system that survives that kind of surge without buckling? This lesson works through the core distributed-systems principles, the CAP theorem, sharding, caching, and event-driven architecture, entirely through the lens of building a real-time cricket scoring platform. By treating every ball as an event flowing through queues, caches, and databases, you will see how abstract theory maps directly onto the very real engineering choices behind the scoreboards millions of fans stare at during a tense run chase.

Core Concepts

The CAP theorem states that during a network partition a distributed system can guarantee either Consistency or Availability, but not both. Map this to live cricket scores. A Consistency-first (CP) system ensures every fan worldwide sees the exact same score at the same instant, but if a network partition splits the cluster, the affected nodes must refuse requests rather than serve a possibly wrong score, meaning the scoreboard may go dark, reminiscent of Cricinfo buckling under load during the 2011 World Cup. An Availability-first (AP) system keeps the scoreboard responsive for everyone even during a partition, accepting that some users may briefly see a slightly stale score (say 247/4 while the truth is 251/4) until the system reconverges. Most live-score platforms lean AP for the public read path, fans tolerate a one-second-stale boundary far better than a blank screen, while keeping the authoritative scorer's write path strongly consistent.

```
bash
```

```
Mobile / Web Clients
```

```
|
```

```
v
```

```
[ CDN ]
```

```
(AP: serves cached scoreboards, edge-close,  
stale-while-revalidate for availability)
```

```
|
```

```
v
```

```
[ API Gateway ]
```

```
(auth, rate limiting, routing)
```

```
|
```

```
v
```

```
[ Score Update Service ]
```

```
(CP write path: the official scorer's  
ball entries are validated, ordered)
```

```
|
```

```
v
```

```
[ Message Queue: Kafka ]
```

```
(durable, ordered event log per match;  
decouples writers from readers)
```

```
|
```

```
v
```

```
[ Redis Cache ]
```

```
(AP: fast reads,  
pub/sub live push,  
eventually consistent)
```

```
v
```

```
[ PostgreSQL ]
```

```
(CP: durable system of record,  
strongly consistent match history,  
replicated with quorum writes)
```

Each hop makes a deliberate CAP trade-off. The CDN and Redis cache prioritise availability and low latency on the read-heavy fan path, tolerating brief staleness. Kafka provides a durable, strictly ordered log so balls are never lost or reordered, the backbone of consistency. PostgreSQL is the consistent system of record for match history. Latency compounds across hops: a CDN edge hit returns in single-digit milliseconds, a Redis read in roughly one millisecond, while a cold PostgreSQL query plus Kafka round trip can add tens of milliseconds, which is exactly why hot scorecards are served from cache.

How It Works Under the Hood

Suppose ten IPL matches run on the same evening of a packed schedule. Naively routing all of them to one database node creates a hotspot, so we shard match data across nodes using consistent hashing. Each database node is placed at multiple points on a hash ring, and each match id is hashed to a point; the match is owned by the first node encountered clockwise. Virtual nodes (multiple ring positions per physical node) smooth out the distribution so no single node is overloaded. A replication factor of three means each match also lands on the next two nodes for fault tolerance. The key benefit appears when you add a new DB node mid-tournament: only the keys in the arc immediately before the new node's positions migrate, rather than rehashing everything. This avoids a

thundering herd of cache and data movement that a simple modulo-N scheme would trigger every time the cluster size changes.

python

```
import hashlib
import bisect

class ConsistentHashRing:
    def __init__(self, virtual_nodes: int = 100):
        self.virtual_nodes = virtual_nodes
        self._ring = {}          # hash → node
        self._sorted_hashes = [] # sorted ring positions

    def _hash(self, key: str) → int:
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def add_node(self, node: str) → None:
        for i in range(self.virtual_nodes):
            h = self._hash(f"{node}#{i}")
            self._ring[h] = node
            bisect.insort(self._sorted_hashes, h)

    def remove_node(self, node: str) → None:
        for i in range(self.virtual_nodes):
            h = self._hash(f"{node}#{i}")
            self._ring.pop(h, None)
            idx = bisect.bisect_left(self._sorted_hashes, h)
            if idx < len(self._sorted_hashes) and self._sorted_hashes[idx] == h:
                self._sorted_hashes.pop(idx)

    def get_node(self, match_id: str) → str:
        if not self._ring:
            raise RuntimeError("No DB nodes available")
        h = self._hash(match_id)
        idx = bisect.bisect(self._sorted_hashes, h) % len(self._sorted_hashes)
        return self._ring[self._sorted_hashes[idx]]

if __name__ == "__main__":
    ring = ConsistentHashRing()
    for node in ("db-node-1", "db-node-2", "db-node-3"):
        ring.add_node(node)
```

```
matches = ["IPL-MI-vs-CSK", "IPL-RCB-vs-KKR", "IPL-SRH-vs-DC",  
           "IPL-GT-vs-RR", "IPL-PBKS-vs-LSG"]  
  
for m in matches:  
    print(f"{m:20} → {ring.get_node(m)}")
```

💡 Pro Tip: Live cricket data is overwhelmingly read-heavy, score queries vastly outnumber score updates, since one scorer writes a ball but millions read it. Use a write-through cache: when the Score Update Service writes a new ball, it updates Redis and the database together, then issues a Redis PUBLISH on a per-match channel. Connected clients (or fan-out servers holding WebSockets) SUBSCRIBE to that channel and receive the new score pushed instantly, eliminating wasteful polling where millions of phones hammer your API every second asking 'any change yet?'

Common Patterns & Best Practices

CQRS (Command Query Responsibility Segregation) separates the write side from the read side. The official scorer issues commands that append to an event log, while millions of fans query a denormalised, display-optimised read view that is cheap to serve. Event Sourcing makes every ball an immutable event, BallDelivered, BoundaryHit, WicketFallen, and the scorecard becomes a projection computed by replaying those events. This gives you a perfect audit trail and the ability to rebuild any past state. A Circuit Breaker wraps calls to external dependencies such as a third-party Cricinfo data provider, tripping open after repeated failures so a flaky upstream does not cascade into your whole platform. Finally, a Saga coordinates the long-running match lifecycle as a sequence of state transitions, toss, then innings one, innings two, and result, with compensating actions if any step fails.

python

```
import time

class CircuitBreakerOpen(Exception):
    pass

class CircuitBreaker:
    def __init__(self, failure_threshold: int = 3, reset_timeout: float = 10.0):
        self.failure_threshold = failure_threshold
        self.reset_timeout = reset_timeout
        self.failure_count = 0
        self.state = "CLOSED"          # CLOSED | OPEN | HALF_OPEN
        self.opened_at = 0.0

    def call(self, func, *args, **kwargs):
        if self.state == "OPEN":
            if time.time() - self.opened_at ≥ self.reset_timeout:
                self.state = "HALF_OPEN"
            else:
                raise CircuitBreakerOpen("Cricinfo provider unavailable")

        try:
            result = func(*args, **kwargs)
        except Exception:
            self._on_failure()
            raise
        else:
            self._on_success()
            return result

    def _on_success(self) → None:
        self.failure_count = 0
        self.state = "CLOSED"

    def _on_failure(self) → None:
        self.failure_count += 1
        if self.failure_count ≥ self.failure_threshold:
            self.state = "OPEN"
            self.opened_at = time.time()
```



```

if __name__ == "__main__":
    attempts = {"n": 0}

    def fetch_live_score(match_id: str) → str:
        attempts["n"] += 1
        if attempts["n"] ≤ 3:
            raise ConnectionError("provider timeout")
        return f"{match_id}: 187/4 (18.2 ov)"

    breaker = CircuitBreaker(failure_threshold=3, reset_timeout=0.5)
    for _ in range(3):
        try:
            breaker.call(fetch_live_score, "MI-vs-CSK")
        except Exception as e:
            print(f"{breaker.state}: {e}")

```

⚠ Warning: Do not model cricket events as mutable database rows that you overwrite, for example `UPDATE scorecard SET runs = runs + 4`. If you do, you lose the audit trail and can never answer 'what did the score look like after over 12?' or correct a scorer's mistake cleanly. Instead, model each ball as an immutable, append-only event (`BallDelivered` with its runs, extras, and dismissal) and derive the live scorecard as a read projection by folding over those events. You gain replayability, easy debugging, and a complete history of the innings.

Real-World Application

These patterns are not academic. ESPNcricinfo serves cached, denormalised scorecards through CDNs for its read-heavy traffic. Disney+ Hotstar famously handled a record 25.3 million concurrent viewers during the IPL 2023 final, combining adaptive bitrate video streaming with a separate low-latency score-push pipeline so the on-screen scoreboard stays fresh even when the video buffers. Dream11 leans on Redis for real-time fantasy leaderboards, recomputing millions of users' points as wickets fall and boundaries fly. Each of them uses caching, event-driven fan-out, and careful CAP trade-offs exactly as described here.

KEY POINTS

- CAP forces a partition-time choice between consistency and availability; public live-score read paths usually favour availability with brief staleness.
- Consistent hashing with virtual nodes distributes match data evenly and migrates only a small key range when nodes join or leave.
- Write-through caching plus Redis pub/sub pushes new scores to clients instantly, eliminating wasteful polling at massive read volumes.
- CQRS separates the scorer's write path from the fan's read path so each can be optimised and scaled independently.
- Event sourcing stores each ball as an immutable event, giving a full audit trail and letting the scorecard be a replayable projection.
- Circuit breakers and sagas keep the platform resilient: isolating flaky data providers and coordinating the toss-to-result match lifecycle safely.

Q: During a critical India vs Australia Test match final day, your score database partition occurs. Your system is CP. What does this mean for users?

- A** All users keep seeing scores, but some scores may be slightly stale until the partition heals
- B** The system sacrifices availability on the affected partition: requests that cannot guarantee a consistent score are rejected or error out until the partition heals
- C** The system silently shows different scores to different users with no errors
- D** The partition has no effect because CP systems never go down

Explanation: A CP system chooses consistency over availability during a partition. Rather than risk returning a stale or divergent score, nodes that cannot reach quorum refuse to serve requests, so affected users see errors or timeouts until the partition heals. This guarantees no one ever sees a wrong score, but availability suffers. Option A describes AP behaviour, and C and D misstate how partitions and CP systems work.

Q: Your cricket live score API handles 2M req/s during peak. Scores update every 30 seconds. Which caching strategy gives lowest latency while keeping data fresh?

- A** Cache-aside with a 30-minute TTL so most reads hit the database
- B** No caching, query PostgreSQL directly on every request for guaranteed freshness
- C** Write-through cache updated on each score change, fronted by a CDN with a short TTL (around 30s) plus push invalidation, so reads hit the edge and stay fresh
- D** A 1-hour CDN TTL with no invalidation to maximise cache hit ratio

Explanation: Because updates occur roughly every 30 seconds and reads dominate at 2M req/s, you want reads served from a cache or CDN edge for the lowest latency, while keeping data fresh around the update cadence. A write-through cache refreshed on each score change, fronted by a CDN with a short TTL plus active invalidation, achieves both. Long TTLs (A, D) serve stale scores, and no caching (B) cannot survive 2M req/s.

AWS Solutions Architect: Capstone Mini Project

Topic: AWS Solutions Architect

Project Overview

In this capstone you will build a production-grade serverless cricket scoring API on AWS, the kind of back-end that could power a club cricket tracker or a school tournament scorer. The API fetches live scores, stores match history, and sends push notifications the moment a wicket falls. Along the way you will use AWS Lambda for compute, API Gateway for HTTP routing, DynamoDB for storage, SNS for notifications, and S3 plus CloudFront to host a static scoreboard frontend, all secured with least-privilege IAM. The architecture scales automatically and a cricket fan who learns these services can run this for their local cricket club essentially for free within the AWS free tier, turning a weekend project into a real tool used every match day.

Learning Objectives

KEY POINTS

- Understand Lambda cold starts and apply warm-up strategies (provisioned concurrency, lightweight handlers) for fast scorecard responses.
- Apply DynamoDB single-table design, modelling ball-by-ball data and aggregated scorecards under one partition key.
- Configure API Gateway stages, throttling, and CORS so the scoreboard frontend can call the API safely.
- Implement the SNS fan-out pattern to broadcast wicket alerts to email and SMS subscribers simultaneously.
- Host a static scoreboard on S3 behind CloudFront with sensible cache-control headers.
- Write least-privilege IAM roles granting each Lambda only the DynamoDB actions it actually needs.

Technical Requirements

KEY POINTS

- POST /matches creates a new match with team names, venue, and format, returning a generated matchId.
- PUT /matches/{id}/ball records a single ball delivery (bowler, batsman, runs, extras, dismissal).
- GET /matches/{id}/scorecard returns the aggregated scorecard for a match.
- GET /matches returns recent matches ordered by start time using a GSI.
- An SNS notification fires whenever a wicket falls, with batsman, method, score, and over.
- DynamoDB TTL automatically expires old match data after 90 days to control storage cost.
- p99 latency for GET scorecard stays under 100ms via single-item reads and warm Lambdas.
- Daily cost stays under \$1 for 10,000 matches/day, staying within or near the AWS free tier.

Architecture & Design

The heart of the design is a single DynamoDB table using composite keys. Each match's ball-by-ball events use PK=MATCH#{matchId} and SK=BALL#{over}#{ball}, while the aggregated view lives under the same PK with SK=SCORECARD, so one query by partition key retrieves the whole match cheaply. API Gateway routes each endpoint to a dedicated Lambda function, which reads or writes DynamoDB. An SNS topic named CricketEvents fans out wicket alerts to SMS and email subscribers. The static scoreboard (HTML and JS) sits in an S3 bucket served through a CloudFront distribution with cache-control headers for fast global delivery. Each Lambda assumes an IAM role scoped to the minimum DynamoDB actions it needs, such as ddb:GetItem and ddb:PutItem only. Serverless is the right fit here because cricket traffic is spiky: an obscure club game generates almost nothing, but an IPL-style fixture can jump from zero to two million requests in minutes, and Lambda plus DynamoDB on-demand absorb that surge automatically without pre-provisioning servers.

yaml

```
AWSTemplateFormatVersion: '2010-09-09'
Transform: AWS::Serverless-2016-10-31
Description: Serverless live cricket score API
```

Globals:

Function:

```
Runtime: python3.12
Timeout: 10
MemorySize: 256
Environment:
  Variables:
    TABLE_NAME: !Ref CricketTable
    EVENTS_TOPIC: !Ref CricketEventsTopic
```

Resources:

CricketTable:

```
Type: AWS::DynamoDB::Table
Properties:
  TableName: CricketMatches
  BillingMode: PAY_PER_REQUEST
  AttributeDefinitions:
```

- AttributeName: PK
AttributeType: S
- AttributeName: SK
AttributeType: S
- AttributeName: GSI1PK
AttributeType: S
- AttributeName: startTime
AttributeType: S

KeySchema:

- AttributeName: PK
KeyType: HASH
- AttributeName: SK
KeyType: RANGE

GlobalSecondaryIndexes:

- IndexName: RecentMatches
KeySchema:
 - AttributeName: GSI1PK
KeyType: HASH

```

      - AttributeName: startTime
        KeyType: RANGE
      Projection:
        ProjectionType: ALL
    TimeToLiveSpecification:
      AttributeName: ttl
      Enabled: true

CricketEventsTopic:
  Type: AWS::SNS::Topic
  Properties:
    TopicName: CricketEvents

WicketEmailSubscription:
  Type: AWS::SNS::Subscription
  Properties:
    TopicArn: !Ref CricketEventsTopic
    Protocol: email
    Endpoint: club-captain@example.com

CricketApi:
  Type: AWS::Serverless::Api
  Properties:
    StageName: prod
    Cors:
      AllowMethods: "'GET,POST,PUT,OPTIONS'"
      AllowHeaders: "'Content-Type'"
      AllowOrigin: "'*'"

CreateMatchFunction:
  Type: AWS::Serverless::Function
  Properties:
    Handler: app.create_match
    CodeUri: src/
    Policies:
      - DynamoDBWritePolicy:
          TableName: !Ref CricketTable
    Events:
      Api:
        Type: Api
        Properties:
          RestApiId: !Ref CricketApi

```

Path: /matches

Method: post

RecordBallFunction:

Type: AWS::Serverless::Function

Properties:

Handler: app.record_ball

CodeUri: src/

Policies:

- DynamoDBCrudPolicy:

TableName: !Ref CricketTable

- SNSPublishMessagePolicy:

TopicName: !GetAtt CricketEventsTopic.TopicName

Events:

Api:

Type: Api

Properties:

RestApiId: !Ref CricketApi

Path: /matches/{id}/ball

Method: put

GetScorecardFunction:

Type: AWS::Serverless::Function

Properties:

Handler: app.get_scorecard

CodeUri: src/

Policies:

- DynamoDBReadPolicy:

TableName: !Ref CricketTable

Events:

Api:

Type: Api

Properties:

RestApiId: !Ref CricketApi

Path: /matches/{id}/scorecard

Method: get

ListMatchesFunction:

Type: AWS::Serverless::Function

Properties:

Handler: app.list_matches

CodeUri: src/


```

Policies:
  - DynamoDBReadPolicy:
      TableName: !Ref CricketTable

Events:
  Api:
    Type: Api
    Properties:
      RestApiId: !Ref CricketApi
      Path: /matches
      Method: get

ScoreboardBucket:
  Type: AWS::S3::Bucket
  Properties:
    BucketName: cricket-scoreboard-static

ScoreboardCDN:
  Type: AWS::CloudFront::Distribution
  Properties:
    DistributionConfig:
      Enabled: true
      DefaultRootObject: index.html
      Origins:
        - Id: S3Origin
          DomainName: !GetAtt ScoreboardBucket.RegionalDomainName
          S3OriginConfig: {}
      DefaultCacheBehavior:
        TargetOriginId: S3Origin
        ViewerProtocolPolicy: redirect-to-https
        CachePolicyId: 658327ea-f89d-4fab-a63d-7e88639e58f6

Outputs:
  ApiUrl:
    Value: !Sub "https://${CricketApi}.execute-api.${AWS::Region}.amazonaws.com/prod"
  CdnUrl:
    Value: !GetAtt ScoreboardCDN.DomainName

```

Phase 1 — Core Implementation

Phase 1 builds the write path: creating a match and recording ball-by-ball data. Thanks to single-table design, each delivery is one PutItem under PK=MATCH#{matchId} with all its details (bowler,

batsman, runs, extras, dismissal type). Rather than running a separate aggregation job, we keep a SCORECARD item updated atomically with each ball using a DynamoDB UpdateExpression, so the MVP can compute the live scorecard directly from that running aggregate. When a delivery is a wicket, we publish to SNS so subscribers are alerted immediately.

python

```
import json
import os
import time
import uuid
import boto3

dynamodb = boto3.resource("dynamodb")
sns = boto3.client("sns")
table = dynamodb.Table(os.environ["TABLE_NAME"])
EVENTS_TOPIC = os.environ.get("EVENTS_TOPIC")

TTL_SECONDS = 90 * 24 * 60 * 60 # 90 days

def create_match(event, context):
    body = json.loads(event["body"])
    match_id = str(uuid.uuid4())[:8]
    start_time = body.get("startTime", str(int(time.time())))
    table.put_item(Item={
        "PK": f"MATCH#{match_id}",
        "SK": "SCORECARD",
        "GSI1PK": "MATCH",
        "startTime": start_time,
        "team1": body["team1"],
        "team2": body["team2"],
        "venue": body.get("venue", "TBD"),
        "format": body.get("format", "T20"),
        "totalRuns": 0,
        "wickets": 0,
        "balls": 0,
        "ttl": int(time.time()) + TTL_SECONDS,
    })
    return _response(201, {"matchId": match_id})

def record_ball(event, context):
    match_id = event["pathParameters"]["id"]
    b = json.loads(event["body"])
    # Example delivery: Shaheen Afridi to Rohit Sharma, over 14.3
    over = int(b["over"]) # 14
```

```

ball = int(b["ball"])          # 3
runs = int(b.get("runs", 0))
extras = int(b.get("extras", 0))
is_wicket = bool(b.get("is_wicket", False))

table.put_item(Item={
    "PK": f"MATCH#{match_id}",
    "SK": f"BALL#{over:03d}#{ball}",
    "bowler": b["bowler"],
    "batsman": b["batsman"],
    "runs": runs,
    "extras": extras,
    "dismissal": b.get("dismissal"),
    "is_wicket": is_wicket,
    "ttl": int(time.time()) + TTL_SECONDS,
})

# Atomically update the running scorecard aggregate.
table.update_item(
    Key={"PK": f"MATCH#{match_id}", "SK": "SCORECARD"},
    UpdateExpression=(
        "SET totalRuns = totalRuns + :r, balls = balls + :one "
        "ADD wickets :w"
    ),
    ExpressionAttributeValues={
        ":r": runs + extras,
        ":one": 1,
        ":w": 1 if is_wicket else 0,
    },
)

if is_wicket and EVENTS_TOPIC:
    sns.publish(
        TopicArn=EVENTS_TOPIC,
        Subject="Wicket!",
        Message=(
            f"WICKET! {b['batsman']} {b.get('dismissal','out')} "
            f"b {b['bowler']} in over {over}.{ball}"
        ),
    )

return _response(200, {"recorded": f"{over}.{ball}"})

```

```
def _response(status, body):
    return {
        "statusCode": status,
        "headers": {"Content-Type": "application/json",
                    "Access-Control-Allow-Origin": "*"},
        "body": json.dumps(body),
    }
```

Phase 2 — Feature Completion

Phase 2 completes the read side and notifications. The GET scorecard endpoint fetches the aggregated SCORECARD item and formats batting and bowling figures, computing run rate and, for a second innings, the required run rate. The match listing uses the RecentMatches GSI to return matches sorted by start time. We also enrich the SNS wicket message so subscribers receive the dismissed batsman, the method, the current score, and the over.

python

```
from boto3.dynamodb.conditions import Key

def get_scorecard(event, context):
    match_id = event["pathParameters"]["id"]
    resp = table.get_item(
        Key={"PK": f"MATCH#{match_id}", "SK": "SCORECARD"}
    )
    item = resp.get("Item")
    if not item:
        return _response(404, {"error": "match not found"})

    balls = int(item["balls"])
    overs = balls / 6 if balls else 0
    run_rate = round(int(item["totalRuns"]) / overs, 2) if overs else 0.0

    scorecard = {
        "matchId": match_id,
        "teams": f"{item['team1']} vs {item['team2']}",
        "score": f"{item['totalRuns']}/{item['wickets']}",
        "overs": f"{balls // 6}.{balls % 6}",
        "runRate": run_rate,
    }

    target = item.get("target")
    if target:
        runs_needed = int(target) - int(item["totalRuns"])
        balls_left = (20 * 6) - balls # T20 example
        rrr = round(runs_needed / (balls_left / 6), 2) if balls_left else 0.0
        scorecard["requiredRunRate"] = rrr

    return _response(200, scorecard)

def list_matches(event, context):
    resp = table.query(
        IndexName="RecentMatches",
        KeyConditionExpression=Key("GSI1PK").eq("MATCH"),
        ScanIndexForward=False, # newest first
        Limit=20,
```

```

    )
    matches = [
        {"matchId": i["PK"].split("#")[1],
         "teams": f"{i['team1']} vs {i['team2']}",
         "score": f"{i['totalRuns']}/{i['wickets']}"
        }
        for i in resp.get("Items", [])
    ]
    return _response(200, {"matches": matches})

def format_sns_wicket_message(batsman, method, bowler, score, over, ball):
    return (
        f"WICKET! \U0001F3CF\n"
        f"{batsman} {method} b {bowler}\n"
        f"Score: {score} after {over}.{ball} overs"
    )

```

Phase 3 — Polish & Production Readiness

Phase 3 hardens the service. We validate input strictly (balls per over, runs in a legal range, valid dismissal types) and return precise HTTP errors: 404 when a match is not found and 422 for an invalid delivery. We emit structured logs keyed by match_id, over, and ball so CloudWatch Logs Insights queries are easy, publish custom CloudWatch metrics for API errors, and enable X-Ray tracing to pinpoint cold-start latency. A dead-letter queue captures failed SNS deliveries, and DynamoDB point-in-time recovery protects match history durability.

python

```
import json
import logging
from typing import Optional
from pydantic import BaseModel, field_validator, ValidationError
from aws_xray_sdk.core import xray_recorder

logger = logging.getLogger()
logger.setLevel(logging.INFO)

VALID_DISMISSALS = {
    "bowled", "caught", "lbw", "run out", "stumped",
    "hit wicket", "caught and bowled",
}

class BallDelivery(BaseModel):
    bowler: str
    batsman: str
    over: int
    ball: int
    runs: int = 0
    extras: int = 0
    is_wicket: bool = False
    dismissal: Optional[str] = None

    @field_validator("ball")
    @classmethod
    def valid_ball(cls, v):
        if not 1 ≤ v ≤ 6:
            raise ValueError("ball must be between 1 and 6")
        return v

    @field_validator("runs")
    @classmethod
    def valid_runs(cls, v):
        if not 0 ≤ v ≤ 6:
            raise ValueError("runs off the bat must be 0-6")
        return v

    @field_validator("dismissal")
```



```

@classmethod
def valid_dismissal(cls, v):
    if v is not None and v not in VALID_DISMISSALS:
        raise ValueError(f"unknown dismissal: {v}")
    return v

def traced_record_ball(event, context):
    match_id = event["pathParameters"]["id"]
    with xray_recorder.in_subsegment("validate_and_record"):
        try:
            delivery = BallDelivery(**json.loads(event["body"]))
        except ValidationError as e:
            logger.warning(json.dumps({
                "match_id": match_id, "error": "validation",
                "detail": e.errors(),
            }))
            return _response(422, {"error": "invalid delivery",
                                   "detail": e.errors()})

        logger.info(json.dumps({
            "match_id": match_id,
            "over": delivery.over,
            "ball": delivery.ball,
            "event": "ball_recorded",
        }))
        # ... persist via record_ball logic ...
        return _response(200, {"recorded": f"{delivery.over}.{delivery.ball}"})

```

Evaluation Rubric

KEY POINTS

- DynamoDB design quality: correct single-table modelling with composite keys and a GSI for recent-match listing.
- IAM least privilege: each Lambda granted only the specific DynamoDB and SNS actions it requires, nothing broader.
- Error handling completeness: 404 for missing matches, 422 for invalid deliveries, and graceful handling of malformed input.
- SNS integration: wicket events reliably fan out to subscribers with a rich, informative message and a DLQ for failures.
- Cold start optimisation: lightweight handlers, right-sized memory, and provisioned concurrency where p99 latency matters.
- CloudWatch observability: structured logs keyed by match/over/ball, custom error metrics, and X-Ray tracing enabled.
- Cost estimation accuracy: a defensible calculation showing the design stays under \$1/day for 10k matches within the free tier.

💡 Extension Challenges: 1) Add a WebSocket API Gateway so the scoreboard receives live score pushes the instant a ball is recorded, eliminating client polling. 2) Use Lambda@Edge to localise displayed match start times at the CloudFront edge (IST for Indian fans, GMT for UK viewers) without round-tripping to origin. 3) Build a Dream11-style fantasy points calculator as a separate Lambda triggered by DynamoDB Streams, recomputing user fantasy scores automatically every time a new ball event lands in the table.